System-wide Profiling & Tracing



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



System-wide Profiling & Tracing

- Sometimes, the problems are not tied to an application but rather due to the usage of multiple layers (drivers, application, kernel).
- In that case, it might be useful to analyze the whole stack.
- The kernel already includes a large number of tracepoints that can be recorded using specific tools.
- New tracepoints can also be created statically or dynamically using various mechanisms (kprobes for instance).



kprobes



Kprobes

- Allows to insert breaks at almost any kernel address dynamically and to extract debugging and performance information
- Uses code patching to modify text code to insert calls to specific handlers
  - `kprobes` allows to execute specific handlers when the hooked instruction is executed
  - `kretprobes` will trigger when returning from a function allowing to extract the return value of functions but also display the parameters that were used for the function call
- Needs some basic kernel configuration:
  - CONFIG_KPROBES=y to enable general kprobe support
  - CONFIG_KALLSYMS_ALL=y to allow hooking probes using `<symbol_name>` instead of raw function address
  - CONFIG_KPROBE_EVENTS=y to enable kprobes usage as tracing events in `tracefs`
- At the lowest level, k(ret)probes are manipulated with dedicated kernel APIs, allowing to write our own kprobe tools (eg as kernel modules)
- Can also be used from userspace with `/sys/kernel/tracing/kprobe_events`
- See trace/kprobes for more information



Basic kprobe tracing (1/2)

- Add a kprobe on `do_sys_openat2`:

```
$ echo "p:my_probe do_sys_openat2" > /sys/kernel/tracing/kprobe_events
```

- Add a kprobe in the same function but at a specific offset, and capture some arguments

```
$ echo "p:my_probe_2 do_sys_openat2+0x7c file=%r2" > /sys/kernel/tracing/kprobe_events
```

- Insert a kretprobe

```
$ echo 'r:my_retprobe do_sys_openat2 $retval' > /sys/kernel/tracing/kprobe_events
```



Basic kprobe tracing (2/2)

- Show existing kprobes

```
$ cat /sys/kernel/tracing/kprobe_events
```

- Enable a kprobe (ie: start capturing the corresponding event)

```
$ echo 1 > /sys/kernel/tracing/events/kprobes/my_probe/enable
```

- Get data emitted by kprobes

```
$ cat /sys/kernel/tracing/trace
```

- Delete a kprobe

```
$ echo "-:my_probe" >> /sys/kernel/tracing/kprobe_events
```



perf



perf

- *perf* allows to do a wide range of tracing and recording operations.
- The kernel already contains events and tracepoints that can be used. The list is given using `perf list`.
  - Syscall tracepoints must be explicitly enabled in kernel configuration using CONFIG_FTRACE_SYSCALLS.
- New probes can be created dynamically on any symbols and registers when debug info are not present.
- Tracing functions, recording variables and parameters content using their names will require a kernel compiled with CONFIG_DEBUG_INFO.
- If perf does not find `vmlinux` you have to provide it using `-k <vmlinux>`.



perf example

- List all events that matches `syscalls:*`

```
$ perf list syscalls:*
List of pre-defined events (to be used in -e):

  syscalls:sys_enter_accept                          [Tracepoint event]
  syscalls:sys_enter_accept4                         [Tracepoint event]
  syscalls:sys_enter_access                          [Tracepoint event]
  syscalls:sys_enter_adjtimex_time32                 [Tracepoint event]
  syscalls:sys_enter_bind                            [Tracepoint event]
...
```

- Record all `syscalls:sys_enter_read` events for `sha256sum` command into `perf.data` file.

```
$ perf record -e syscalls:sys_enter_read sha256sum /bin/busybox
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.018 MB perf.data (215 samples) ]
```



perf report example

- Display the collected samples ordered by time spent.

```
$ perf report
Samples: 591  of event 'cycles', Event count (approx.): 393877062
Overhead  Command      Shared Object                     Symbol
  22,88%  firefox-esr  [nvidia]                          [k] _nv031568rm
   3,21%  firefox-esr  ld-linux-x86-64.so.2              [.] __minimal_realloc
   2,00%  firefox-esr  libc.so.6                         [.] __stpncpy_ssse3
   1,86%  firefox-esr  libglib-2.0.so.0.7400.0           [.] g_hash_table_lookup
   1,62%  firefox-esr  ld-linux-x86-64.so.2              [.] _dl_strtoul
   1,56%  firefox-esr  [kernel.kallsyms]                 [k] clear_page_rep
   1,52%  firefox-esr  libc.so.6                         [.] __strncpy_sse2_unaligned
   1,37%  firefox-esr  ld-linux-x86-64.so.2              [.] strncmp
   1,30%  firefox-esr  firefox-esr                       [.] malloc
   1,27%  firefox-esr  libc.so.6                         [.] __GI___strcasecmp_l_ssse3
   1,23%  firefox-esr  [nvidia]                          [k] _nv013165rm
   1,09%  firefox-esr  [nvidia]                          [k] _nv007298rm
   1,03%  firefox-esr  [kernel.kallsyms]                 [k] unmap_page_range
   0,91%  firefox-esr  ld-linux-x86-64.so.2              [.] __minimal_free
```



perf probe

- *perf* allows to create dynamic tracepoints on both kernel functions and user-space functions.
- In order to be able to insert probes, CONFIG_KPROBES must be enabled in the kernel.
  - Note: *libelf* is required to compile *perf* with *probe* command support.
- New dynamic probes can be created and then used using *perf record*.
- Often on embedded platforms, `vmlinux` is not present on the target and thus only symbols and registers can be used.



perf probe examples (1/3)

- List all the kernel symbols that can be probed (no debug info needed):

```
$ perf probe --funcs
```

- Create a new probe on `do_sys_openat2` with *filename* named parameter (debug info required).

```
$ perf probe --vmlinux=vmlinux_file do_sys_openat2 filename:string
Added new event:
  probe:do_sys_openat2 (on do_sys_openat2 with filename:string)
```

- Execute `tail` and capture previously created probe event:

```
$ perf record -e probe:do_sys_openat2 tail /var/log/messages
...
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.003 MB perf.data (19 samples) ]
```



perf probe examples (2/3)

- Display the recorded tracepoints with *perf script*:

```
$ perf script
tail   164 [000]  3552.956573: probe:do_sys_openat2: (c02c3750) filename_string="/etc/ld.so.cache"
tail   164 [000]  3552.956642: probe:do_sys_openat2: (c02c3750) filename_string="/lib/tls/v7l/neon/vfp/libresolv.so.2"
...
```

- Create a new probe to capture the return value from `ksys_read`

```
$ perf probe ksys_read%return \$retval
```

- Execute `sha256sum` and capture previously created probe events:

```
$ perf record -e probe:ksys_read__return sha256sum /etc/fstab
```



perf probe examples (3/3)

- List all probes that have been created:

```
$ perf probe -l
  probe:ksys_read__return (on ksys_read%return with ret)
```

- Remove an existing tracepoint:

```
$ perf probe -d probe:ksys_read__return
```



perf record example

- Record all events for all cpus (system-wide mode):

```
$ perf record -a
^C
```

- Display recorded events from perf.data using `perf script`

```
$ perf script
...
klogd    85 [000]   208.609712:     116584   cycles:          b6dd551c memset+0x2c (/lib/libc.so.6)
klogd    85 [000]   208.609898:     121267   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34 (vmlinux)
klogd    85 [000]   208.610094:     127434   cycles:          c02f3ef4 kmem_cache_alloc+0xd0 (vmlinux)
 perf   130 [000]   208.610311:     132915   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34 (vmlinux)
 perf   130 [000]   208.619831:     143834   cycles:          c0a44cf4 _raw_spin_unlock_irqrestore+0x3c
(vmlinux)
klogd    85 [000]   208.620048:     143834   cycles:          c01a07f8 syslog_print+0x170 (vmlinux)
klogd    85 [000]   208.620241:     126328   cycles:          c0100184 vector_swi+0x44 (vmlinux)
klogd    85 [000]   208.620434:     128451   cycles:          c096f228 unix_dgram_sendmsg+0x46c (vmlinux)
kworker/0:2-mm_    44 [000]   208.620653:     133104   cycles:          c0a44c84 _raw_spin_unlock_irq+0x34
(vmlinux)
 perf   130 [000]   208.620859:     138065   cycles:          c0198460 lock_acquire+0x184 (vmlinux)
...
```



Using perf trace

- `perf trace` captures and displays all tracepoints/events that have been triggered when executing a command

```
$ perf trace -e "net:*" ping -c 1 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
      0.000 ping/37820 net:net_dev_queue(skbaddr: 0xffff97bbc6a17900, len: 98,
        name: "enp34s0")
      0.005 ping/37820 net:net_dev_start_xmit(name: "enp34s0",
        skbaddr: 0xffff97bbc6a17900, protocol: 2048, len: 98,
        network_offset: 14, transport_offset_valid: 1, transport_offset: 34)
      0.009 ping/37820 net:net_dev_xmit(skbaddr: 0xffff97bbc6a17900, len: 98,
        name: "enp34s0")
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.867 ms
```



Using perf top

- `perf top` allows to do a live analysis of the running kernel
- It will sample all function calls and display them ordered by most time consuming one.
- This allows to profile the whole system usage

```
$ perf top Samples: 19K of event 'cycles', 4000 Hz, Event count (approx.): 4571734204 lost: 0/0 drop: 0/0
Overhead  Shared Object                          Symbol
   2,01%  [nvidia]                               [k] _nv023368rm
   0,94%  [kernel]                               [k] __static_call_text_end
   0,89%  [vdso]                                 [.] 0x0000000000000655
   0,81%  [nvidia]                               [k] _nv027733rm
   0,79%  [kernel]                               [k] clear_page_rep
   0,76%  [kernel]                               [k] psi_group_change
   0,70%  [kernel]                               [k] check_preemption_disabled
   0,69%  code                                   [.] 0x000000000623108f
   0,60%  code                                   [.] 0x0000000006231083
   0,59%  [kernel]                               [k] preempt_count_add
   0,54%  [kernel]                               [k] module_get_kallsym
   0,53%  [kernel]                               [k] copy_user_generic_string
```



Using a GUI to display perf data

- `perf report` is the default way to display perf data, directly in the console
- There are also graphical tools to display perf data:
  - Flamegraphs
    - Visualization based on hierarchical stacks
    - Allows to quickly find bottlenecks and explore the call stack
    - Popularized by Brendan Gregg tools which allows to generate flamegraphs from `perf` results.
  - Hotspot software
    - Developed and maintained by KDAB
    - A larger tool able to generate various types of visualizations from a `perf.data` file
    - Can also perform the actual perf recording



Visualizing data with flamegraphs

- Get the flamegraph scripts:

```
git clone https://github.com/brendangregg/FlameGraph fl
```

- Capture data:

```
perf record -g -- sleep 30
```

  - The `-g` option records call stacks for each sample

- Format the data:

```
perf script | ./fl/stackcollapse-perf.pl > out.perf-folded
```

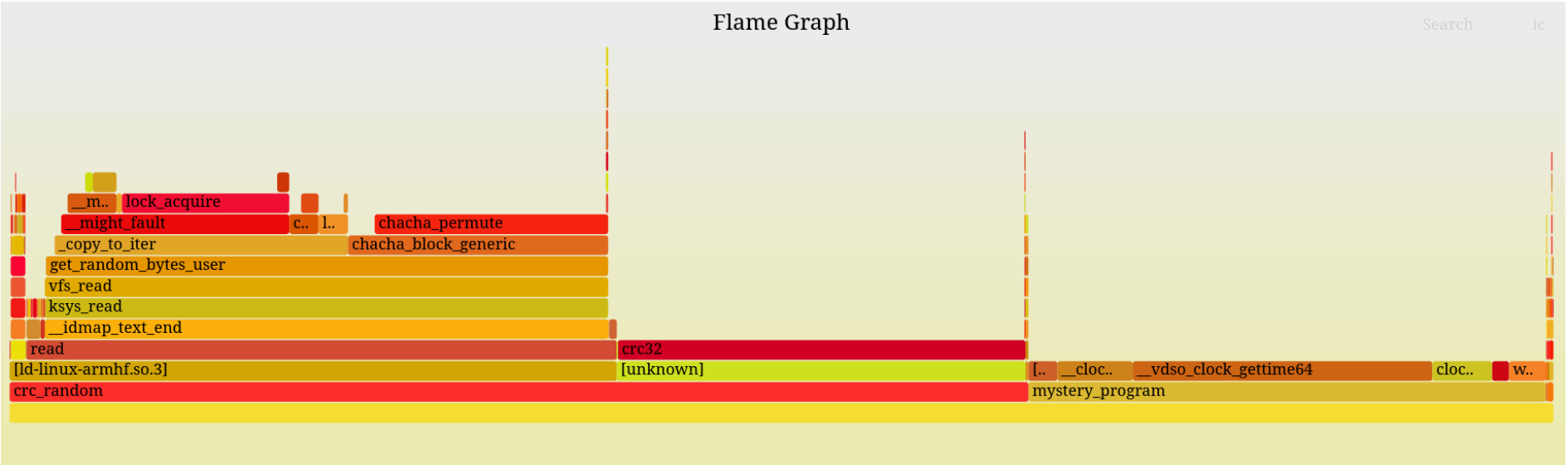
  - Other data sources are supported (eg: DTrace, SystemTap, Intel VTune, gdb...)

- Generate the Flamegraph:

```
./fl/flamegraph.pl out.perf-folded > flamegraph.svg
```

- The flamegraph can then be opened in a web browser

Flamegraph example: CPU flamegraph



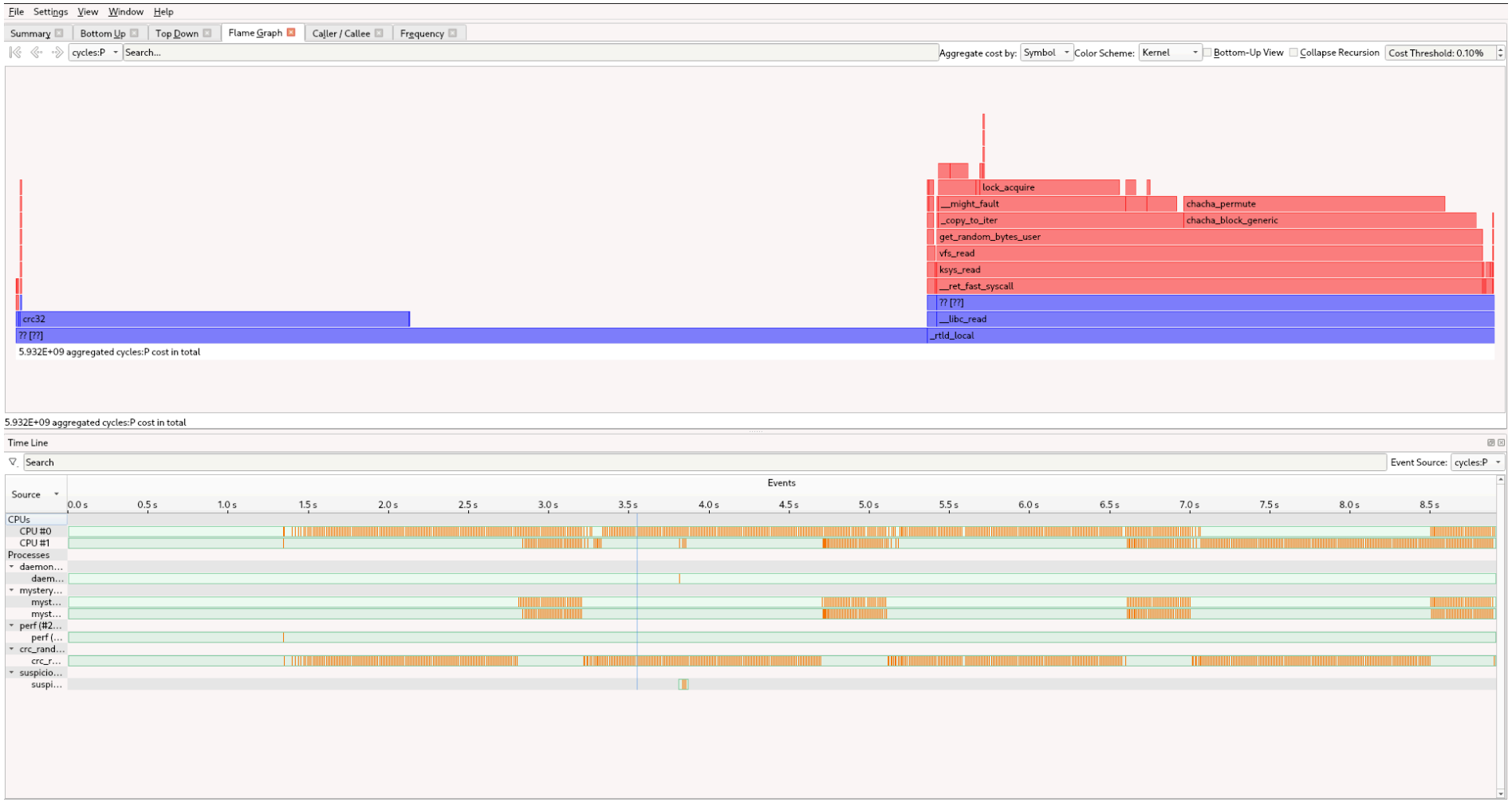
- The plates on top represent the functions sampled by perf during the recording
- The plates width represents how often a function has been sampled by perf
- The plates below represent the call stacks for the sampled functions
- Flamegraphs are interactive: clicking on a plate will zoom on the corresponding callstack
- Colors can be tuned at flamegraph generation (eg: to get a clear split between kernel and userspace)



Visualizing data with hotspot (1/2)

- Designed to provide a frontend to perf data files
- Can generate flamegraphs on the fly, but not only:
  - CPU/tasks timelines
  - Interactive callstacks navigation
  - Code disassembly
- Configurable (eg: allows to set paths to find all needed debug information)

Visualizing data with hotspot (2/2)





ftrace and trace-cmd



ftrace

- *ftrace* is a tracing framework within the kernel which stands for “Function Tracer”.
- It offers a wide range of tracing capabilities allowing to observe the system behavior.
  - Trace static tracepoints already inserted at various locations in the kernel (scheduler, interrupts, etc).
  - Relies on GCC mcount() capability and kernel code patching mechanism to call *ftrace* tracing handlers.
- All traces are recorded in a ring buffer that is optimized for tracing.
- Uses *tracefs* filesystem to control and display tracing events.
  - `# mount -t tracefs nodev /sys/kernel/tracing`.
- *ftrace* support must be enabled in the kernel using CONFIG_FTRACE=y.
- CONFIG_DYNAMIC_FTRACE allows to have a zero overhead tracing support.



ftrace files

- *ftrace* controls are exposed through some specific files located under `/sys/kernel/tracing`.
  - `current_tracer`: Current tracer that is used.
  - `available_tracers`: List of available tracers that are compiled in the kernel.
  - `tracing_on`: Enable/disable tracing.
  - `trace`: Acquired trace in human readable format. Format will differ depending on the tracer used.
  - `trace_pipe`: same as `trace`, but each read consumes the trace as it is read.
  - `trace_marker_raw`: Emit comments from userspace in the trace buffer.
  - `set_ftrace_filter`: Filter some specific functions.
  - `set_graph_function`: Graph only the specified functions child.
- Many other files are exposed, see trace/ftrace.
- *trace-cmd* CLI and *Kernelshark* GUI tools allow to record and visualize tracing data more easily.



ftrace tracers

- ftrace provides several “tracers” which allow to trace different things.
- The tracer to be used should be written to the `current_tracer` file
  - `nop`: Trace nothing, used to disable all tracing.
  - `function`: Trace all kernel functions that are called.
  - `function_graph`: Similar to `function` but traces both entry and exit.
  - `hwlat`: Trace hardware latency.
  - `irqsoff`: Trace sections where interrupts are disabled.
  - `branch`: Trace likely()/unlikely() prediction errors.
  - `mmiotrace`: Trace all accesses to the hardware (`read[bwlq]/write[bwlq]`).
- **Warning: Some tracers can be expensive!**

```
# echo "function" > /sys/kernel/tracing/current_tracer
```



function_graph tracer report example

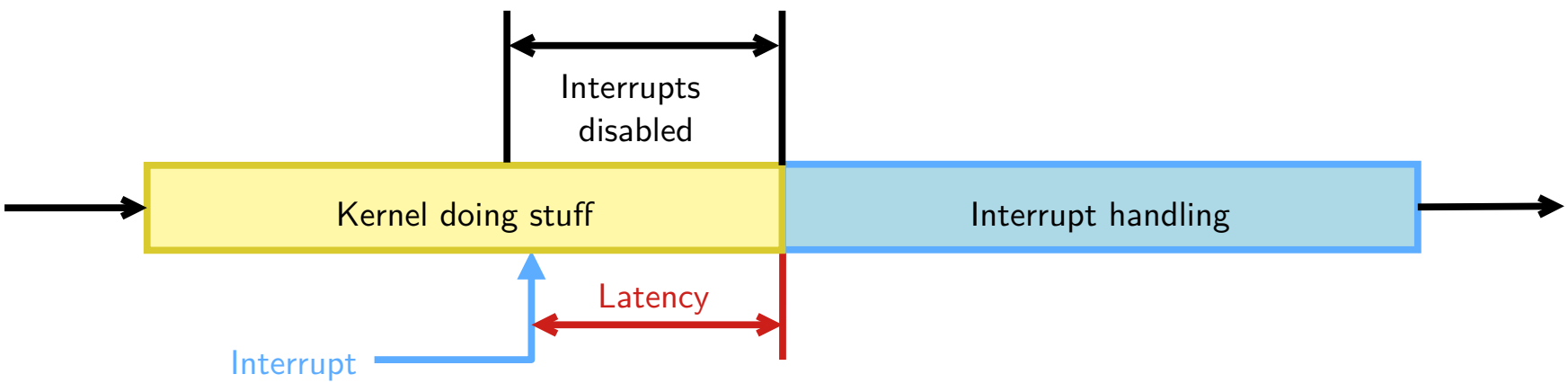
- The *function_graph* traces all the function that executed and their associated callgraphs
- Will display the process, CPU, timestamp and function graph:

```
$ trace-cmd report
...
dd-113   [000]   304.526590: funcgraph_entry:                   |  sys_write() {
dd-113   [000]   304.526597: funcgraph_entry:                   |    ksys_write() {
dd-113   [000]   304.526603: funcgraph_entry:                   |      __fdget_pos() {
dd-113   [000]   304.526609: funcgraph_entry:       6.541 us    |        __fget_light();
dd-113   [000]   304.526621: funcgraph_exit:      + 18.500 us   |      }
dd-113   [000]   304.526627: funcgraph_entry:                   |      vfs_write() {
dd-113   [000]   304.526634: funcgraph_entry:       6.334 us    |        rw_verify_area();
dd-113   [000]   304.526646: funcgraph_entry:       6.208 us    |        write_null();
dd-113   [000]   304.526658: funcgraph_entry:       6.292 us    |        __fsnotify_parent();
dd-113   [000]   304.526669: funcgraph_exit:      + 43.042 us   |      }
dd-113   [000]   304.526675: funcgraph_exit:      + 78.833 us   |    }
dd-113   [000]   304.526680: funcgraph_exit:      + 91.291 us   |  }
dd-113   [000]   304.526689: funcgraph_entry:                   |  sys_read() {
dd-113   [000]   304.526695: funcgraph_entry:                   |    ksys_read() {
dd-113   [000]   304.526702: funcgraph_entry:                   |      __fdget_pos() {
dd-113   [000]   304.526708: funcgraph_entry:       6.167 us    |        __fget_light();
dd-113   [000]   304.526719: funcgraph_exit:      + 18.083 us   |      }
```



irqsoff tracer

- ftrace *irqsoff* tracer allows to trace the irqs latency due to interrupts being disabled for too long.
- Helpful to find why interrupts have high latencies on a system.
- This tracer will record the longest trace with interrupts being disabled.
- This tracer needs to be enabled with CONFIG_IRQSOFF_TRACER=y.
  - `preemptoff`, `premptirqsoff` tracers also exist to trace section of code were preemption is disabled.





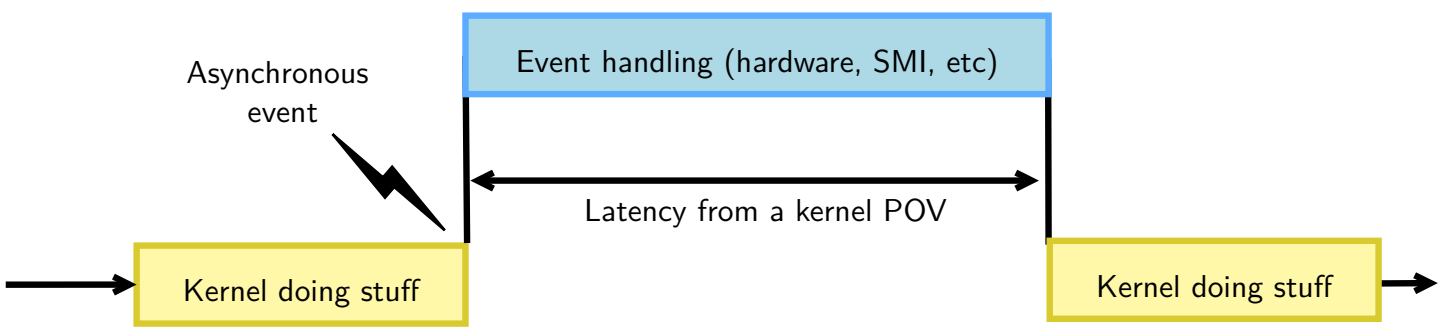
irqsoff: report example

```
# latency: 276 us, #104/104, CPU#0 | (M:preempt VP:0, KP:0, SP:0 HP:0 #P:2)
#    -----------------
#    | task: stress-ng-114 (uid:0 nice:0 policy:0 rt_prio:0)
#    -----------------
#  => started at: __irq_usr
#  => ended at:   irq_exit
#
#
#                    _------=> CPU#
#                   / _-----=> irqs-off
#                  | / _----=> need-resched
#                  || / _---=> hardirq/softirq
#                  ||| / _--=> preempt-depth
#                  |||| /     delay
#  cmd     pid     ||||| time  |   caller
#       /         |||||      |   /
stress-n-114       0d...    2us : __irq_usr
stress-n-114       0d...    7us : gic_handle_irq <-__irq_usr
stress-n-114       0d...   10us : __handle_domain_irq <-gic_handle_irq
...
stress-n-114       0d...  270us : __local_bh_disable_ip <-__do_softirq
stress-n-114       0d.s.  275us : __do_softirq <-irq_exit
stress-n-114       0d.s.  279us+: tracer_hardirqs_on <-irq_exit
stress-n-114       0d.s.  290us : <stack trace>
```



Hardware latency detector

- ftrace *hwlat* tracer will help to find if the hardware generates latency.
  - System Management interrupts for instance are non maskable and directly trigger some firmware support feature, suspending CPU execution.
  - Interrupts handled by secure monitor can also cause this kind of latency.
- If some latency is found with this tracer, the system is probably not suitable for real time usage.
- Uses a single core looping while interrupts are disabled and measuring the time elapsed between two consecutive time reads.
- Needs to be builtin the kernel with CONFIG_HWLAT_TRACER=y.





trace-cmd

- *trace-cmd* is a tool written by Steven Rostedt which allows interacting with *ftrace* (man 1 trace-cmd).
- The tracers supported by *trace-cmd* are those exposed by ftrace.
- *trace-cmd* offers multiple commands:
  - *list*: List available plugins/events that can be recorded.
  - *record*: Record a trace into the file `trace.dat`.
  - *report*: Display `trace.dat` acquisition results.



trace-cmd examples (1/3)

- List available tracers

```
$ trace-cmd list -t
blk mmiotrace function_graph function nop
```

- List available events

```
$ trace-cmd list -e
...
migrate:mm_migrate_pages_start
migrate:mm_migrate_pages
tlb:tlb_flush
syscalls:sys_exit_process_vm_writev
...
```

- List available functions for filtering with `function` and `function_graph` tracers

```
$ trace-cmd list -f
...
wait_for_initramfs
__ftrace_invalid_address___64
calibration_delay_done
calibrate_delay
...
```



trace-cmd examples (2/3)

- Start the function tracer and record data globally on the system

```
$ trace-cmd record -p function
```

- Use the function graph tracer but filter only `spi_*` functions

```
$ trace-cmd record -l spi_* -p function_graph
```

- Run the *irqsoff* tracer on the system:

```
$ trace-cmd record -p irqsoff
```

- Record only `irq_handler_exit/irq_handler_entry` events on the system:

```
$ trace-cmd record -e irq:irq_handler_exit -e irq:irq_handler_entry
```



trace-cmd examples (3/3)

- Visualize the data that have been acquired in `trace.dat`:

```
$ trace-cmd report
```

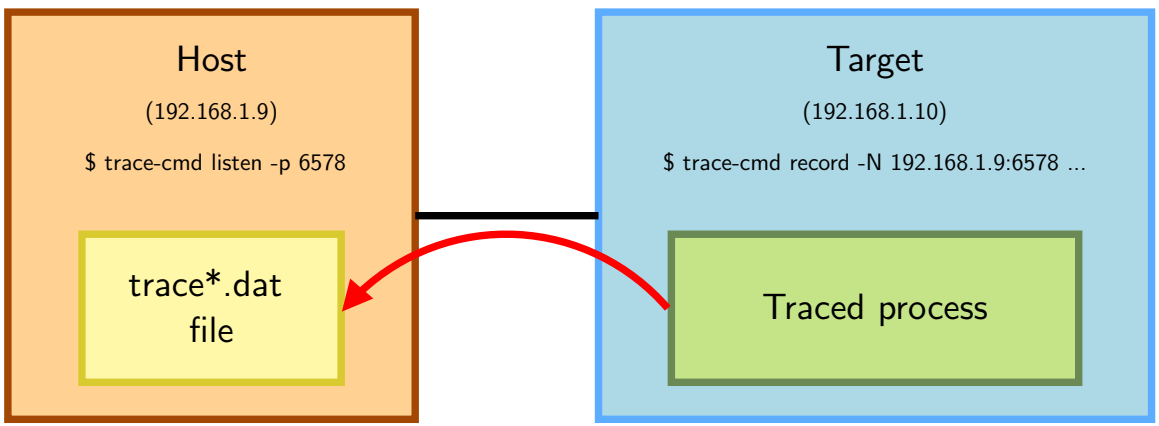
- Reset all the *ftrace* buffers and remove tracers

```
$ trace-cmd reset
```



Remote tracing with trace-cmd

- *trace-cmd* output can be quite big and thus difficult to store on an embedded platform with limited storage.
- For that purpose, a `listen` command is available and allows sending the acquisitions over the network:
  - Run `trace-cmd listen -p 6578` on the host that will be storing the traces
  - On the remote target, use `trace-cmd record -N <host_ip>:6578` to specify the host that will store the traces





trace_printk()

- trace_printk() allows to emit strings in the trace buffer
- Useful to trace some specific conditions in your code and display it in the trace buffer

```
#include <linux/ftrace.h>
void read_hw()
{
  if (condition)
    trace_printk("Condition is true!\n");
}
```

- Will display the following in the trace buffer for `function_graph` tracer

```
1)               |             read_hw() {
1)               |                /* Condition is true! */
1)   2.657 us    |             }
```



Adding ftrace tracepoints (1/2)

- For some custom needs, it might be needed to add custom tracepoints
- First, one needs to declare the tracepoint definition in a `.h` file

```
#undef TRACE_SYSTEM
#define TRACE_SYSTEM subsys

#if !defined(_TRACE_SUBSYS_H) || defined(TRACE_HEADER_MULTI_READ)
#define _TRACE_SUBSYS_H

#include <linux/tracepoint.h>

DECLARE_TRACE(subsys_eventname,
        TP_PROTO(int firstarg, struct task_struct *p),
        TP_ARGS(firstarg, p));

#endif /* _TRACE_SUBSYS_H */

/* This part must be outside protection */
#include <trace/define_trace.h>
```



Adding ftrace tracepoints (2/2)

- Then, emit tracepoint in a `.c` file using that header file

```
#include <trace/events/subsys.h>

#define CREATE_TRACE_POINTS
DEFINE_TRACE(subsys_eventname);

void any_func(void)
{
  ...
  trace_subsys_eventname(arg, task);
  ...
}
```

- See trace/tracepoints for more information



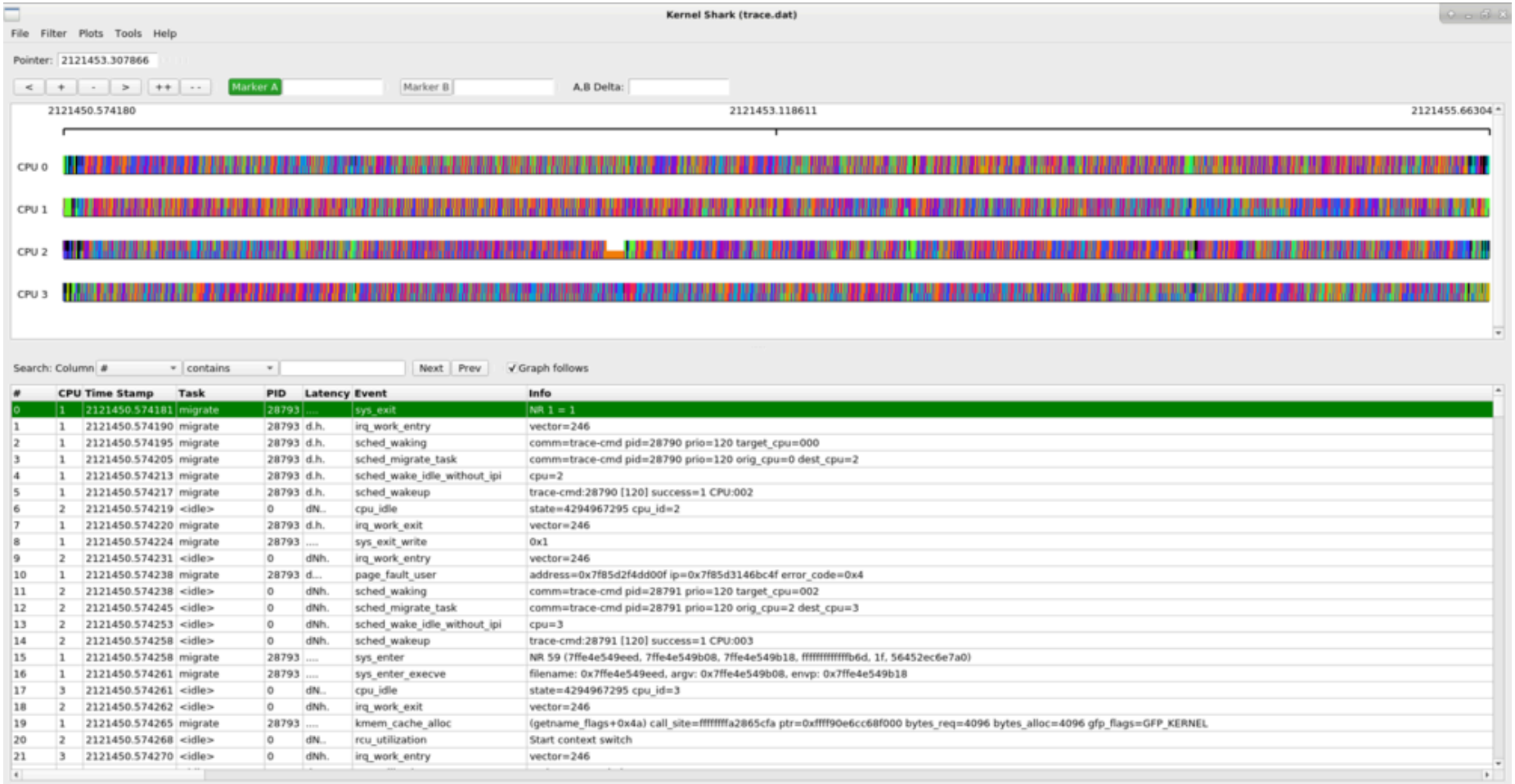
Kernelshark

- Kernelshark is a Qt-based graphical interface for processing *trace-cmd* trace.dat reports.
- Can also setup and acquire data using *trace-cmd*.
- Displays CPU and tasks as different colors along with the recorded events.
- Useful when a deep analysis is required for a specific bug.



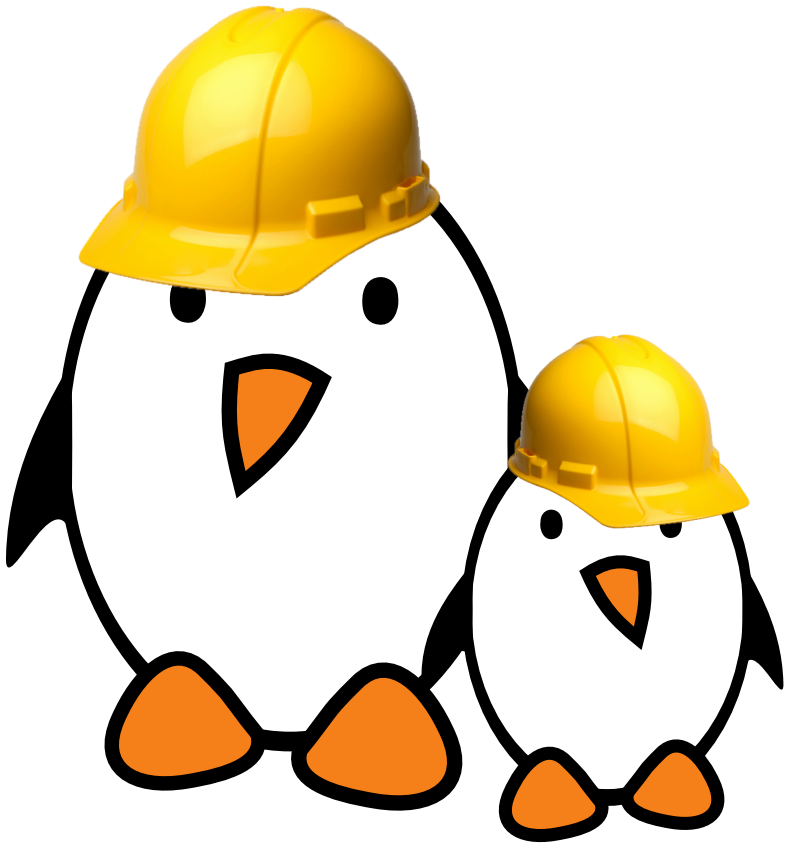


kernelshark



#	CPU	Time Stamp	Task	PID	Latency	Event	Info
0	1	2121450.574181	migrate	28793	....	sys_exit	NR 1 = 1
1	1	2121450.574190	migrate	28793	d.h.	irq_work_entry	vector=246
2	1	2121450.574195	migrate	28793	d.h.	sched_waking	comm=trace-cmd pid=28790 prio=120 target_cpu=000
3	1	2121450.574205	migrate	28793	d.h.	sched_migrate_task	comm=trace-cmd pid=28790 prio=120 orig_cpu=0 dest_cpu=2
4	1	2121450.574213	migrate	28793	d.h.	sched_wake_idle_without_ipi	cpu=2
5	1	2121450.574217	migrate	28793	d.h.	sched_wakeup	trace-cmd:28790 [120] success=1 CPU:002
6	2	2121450.574219	<idle>	0	dN..	cpu_idle	state=4294967295 cpu_id=2
7	1	2121450.574220	migrate	28793	d.h.	irq_work_exit	vector=246
8	1	2121450.574224	migrate	28793	....	sys_exit_write	0x1
9	2	2121450.574231	<idle>	0	dNh.	irq_work_entry	vector=246
10	1	2121450.574238	migrate	28793	d...	page_fault_user	address=0x7f85d2f4dd00f ip=0x7f85d3146bc4f error_code=0x4
11	2	2121450.574238	<idle>	0	dNh.	sched_waking	comm=trace-cmd pid=28791 prio=120 target_cpu=002
12	2	2121450.574245	<idle>	0	dNh.	sched_migrate_task	comm=trace-cmd pid=28791 prio=120 orig_cpu=2 dest_cpu=3
13	2	2121450.574253	<idle>	0	dNh.	sched_wake_idle_without_ipi	cpu=3
14	2	2121450.574258	<idle>	0	dNh.	sched_wakeup	trace-cmd:28791 [120] success=1 CPU:003
15	1	2121450.574258	migrate	28793	....	sys_enter	NR 59 (7ffe4e549eed, 7ffe4e549b08, 7ffe4e549b18, ffffffffffffb6d, 1f, 56452ec6e7a0)
16	1	2121450.574261	migrate	28793	....	sys_enter_execve	filename: 0x7ffe4e549eed, argv: 0x7ffe4e549b08, envp: 0x7ffe4e549b18
17	3	2121450.574261	<idle>	0	dN..	cpu_idle	state=4294967295 cpu_id=3
18	2	2121450.574262	<idle>	0	dNh.	irq_work_exit	vector=246
19	1	2121450.574265	migrate	28793	....	kmem_cache_alloc	(getname_flags+0x4a) call_site=ffffffffa2865cfa ptr=0xffff90e6cc68f000 bytes_req=4096 bytes_alloc=4096 gfp_flags=GFP_KERNEL
20	2	2121450.574268	<idle>	0	dN..	rcu_utilization	Start context switch
21	3	2121450.574270	<idle>	0	dNh.	irq_work_entry	vector=246

Practical lab - System wide profiling



Profiling a system from userspace to kernel space

- Profiling with ftrace, uprobes and kernelshark
- Profiling with perf



LTTng



LTTng

- LTTng is an open source tracing framework for Linux maintained by the EfficiOS company.
- LTTng allows understanding the interactions between the kernel and applications (C, C++, Java, Python).
  - Also expose a `/dev/lttng-logger` that can be used from any application.
- Tracepoints are associated with a payload (data).
- LTTng is focused on low-overhead tracing.
- Uses the Common Trace Format (so traces are readable with other software like babeltrace or trace-compass)





Tracepoints with LTTng

- LTTng works with a session daemon that receive all events from kernel and userspace LTTng tracing components.
- LTTng can use and trace the following instrumentation points:
  - User space LTTng tracepoints
  - Linux user space probes
  - Linux kernel system calls
  - LTTng kernel tracepoints
  - kprobes and kretprobes



Creating userspace tracepoints with LTTng

- New userspace tracepoints can be defined using LTTng.
- Tracepoints have multiple characteristics:
  - A provider namespace
  - A name identifying the tracepoint
  - Parameters of various types (int, char *, etc)
  - Fields describing how to display the tracepoint parameters (decimal, hexadecimal, etc) (see LTTng-ust manpage for types)
- Developers must perform multiple operations to use UST tracepoint: write a tracepoint provider (.h), write a tracepoint package (.c), build the package, call the tracepoint in the traced application, and finally build the application, linked with lttng-ust library and the package provider.
- LTTng provides the `lttng-gen-tp` to ease all those steps, allowing to only write a template (.tp) file.



Defining a LTTng tracepoint (1/2)

- Tracepoint template (`hello_world-tp.tp`):

```
LTTNG_UST_TRACEPOINT_EVENT(
  // Tracepoint provider name
  hello_world,

  // Tracepoint/event name
  my_first_tracepoint,

  // Tracepoint arguments (input)
  LTTNG_UST_TP_ARGS(
      char *, text
  ),

  // Tracepoint/event fields (output)
  LTTNG_UST_TP_FIELDS(
      lttng_ust_field_string(message, text)
  )
)
```

- `lttng-gen-tp` will take this template file and generate/build all needed files (.h, .c and .o files)



Defining a LTTng tracepoint (2/2)

- Build tracepoint provider:

```
$ lttng-gen-tp hello_world-tp.tp
```

- Tracepoint usage (`hello_world.c`):

```c
#include <stdio.h>
#include "hello-tp.h"

int main(int argc, char *argv[])
{
    lttng_ust_tracepoint(hello_world, my_first_tracepoint, "hi there!");
    return 0;
}
```

- Compilation:

```
$ gcc hello_world.c hello_world-tp.o -llttng-ust -o hello_world
```



Using LTTng

```
$ lttng create my-tracing-session --output=./my_traces
$ lttng list --kernel
$ lttng list --userspace
$ lttng enable-event --userspace hello_world:my_first_tracepoint
$ lttng enable-event --kernel --syscall open,close,write
$ lttng start
$ /* Run your application or do something */
$ lttng destroy
$ babeltrace2 ./my_traces
```

- You can also use trace-compass to display the traces in a GUI



Remote tracing with LTTng

- LTTng allows to record traces over the network.
- Useful for embedded systems with limited storage capabilities.
- On the remote computer, run `lttng-relayd` command

```
$ lttng-relayd --output=${PWD}/traces
```

- Then on the target, at session creation, use the `--set-url`

```
$ lttng create my-session --set-url=net://remote-system
```

- Traces will then be recorded directly on the remote computer.



eBPF



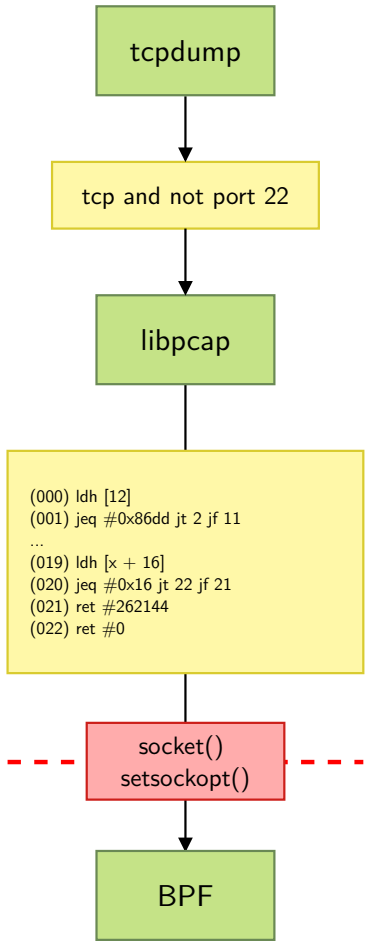
The ancestor: Berkeley Packet filter

- BPF stands for Berkeley Packet Filter and was initially used for network packet filtering
- BPF is implemented and used in Linux to perform Linux Socket Filtering (see networking/filter)
- tcpdump and Wireshark heavily rely on BPF (through libpcap) for packet capture



BPF in libpcap: setup

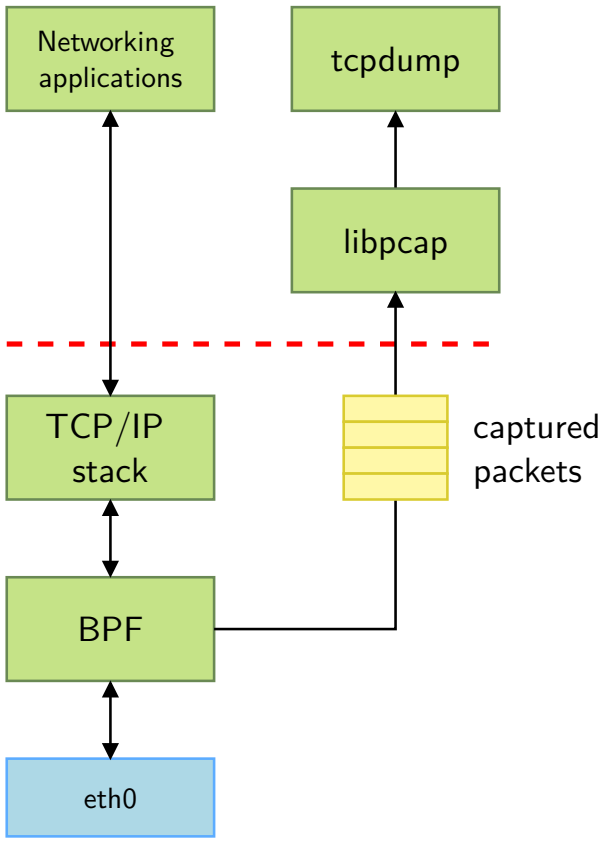
- tcpdump passes the capture filter string from the user to libpcap
- libpcap translates the capture filter into a binary program
  - This program uses the instruction set of an abstract machine (the "BPF instruction set")
- libpcap sends the binary program to the kernel via the `setsockopt()` syscall





BPF in libpcap: capture

- The kernel implements the BPF “virtual machine”
- The BPF virtual machine executes the program for every packet
- The program inspects the packet data and returns a non-zero value if the packet must be captured
- If the return value is non-zero, the packet is captured in addition to regular packet processing





eBPF (1/2)

- eBPF is a new framework allowing to run small user programs directly in the kernel, in a safe and efficient way. It has been added in kernel 3.18 but it is still evolving and receiving updates frequently.
- eBPF programs can capture and expose kernel data to userspace, and also alter kernel behavior based on some user-defined rules.
- eBPF is event-driven: an eBPF program is triggered and executed on a specific kernel event
- A major benefit from eBPF is the possibility to reprogram the kernel behavior, without performing kernel development:
  - no risk of crashing the kernel because of bugs
  - faster development cycles to get a new feature ready



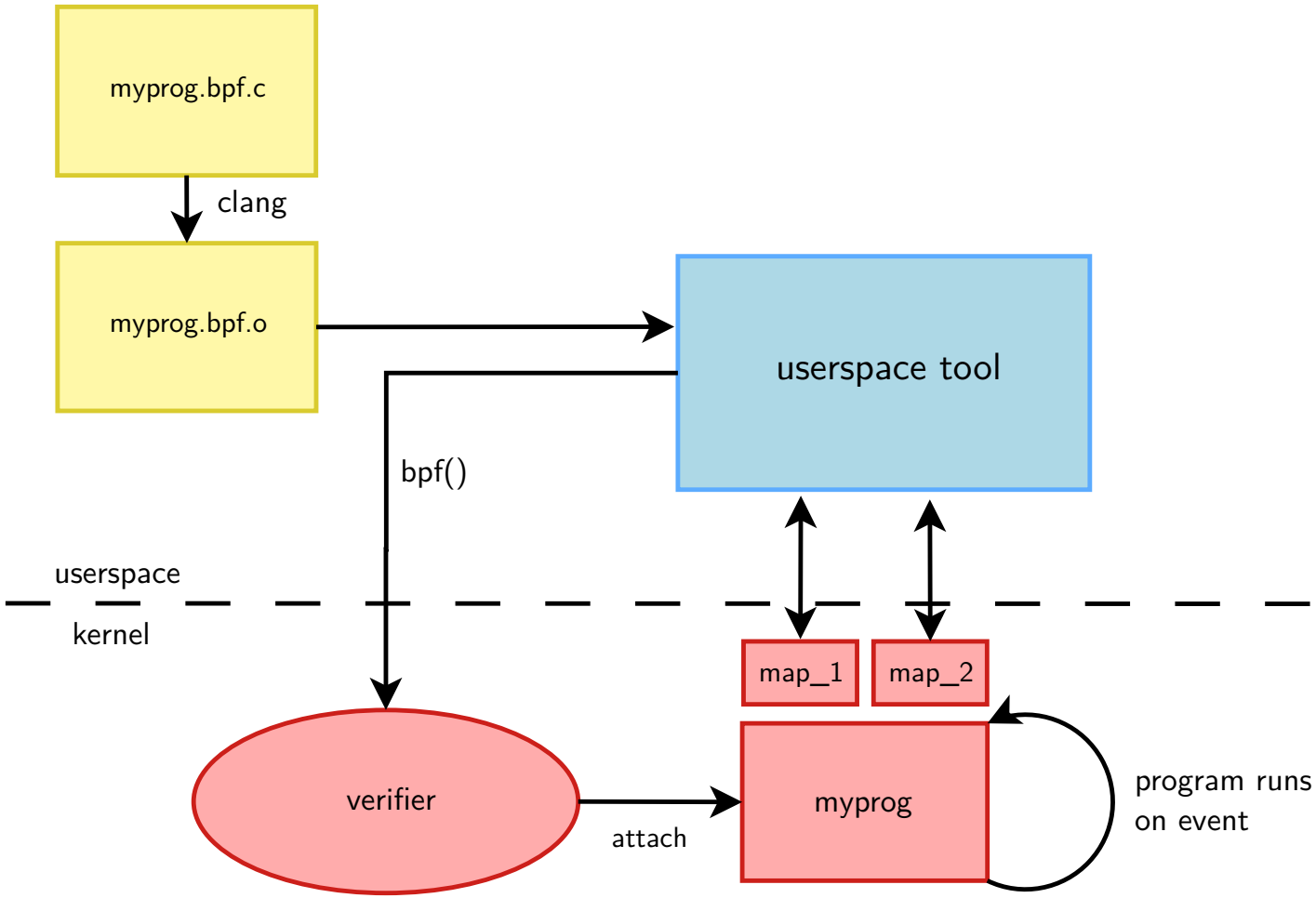
Image credits: https://ebpf.io/



eBPF (2/2)

- The most notable eBPF features are:
  - A new instruction set, interpreter and verifier
  - A wide variety of “attach” locations, allowing to hook programs almost anywhere in the kernel
  - dedicated data structures called “maps”, to exchange data between multiple eBPF programs or between programs and userspace
  - A dedicated `bpf()` syscall to manipulate eBPF programs and data
  - plenty of (kernel) helper functions accessible from eBPF programs.

eBPF program lifecycle





Kernel configuration for eBPF

- CONFIG_NET to enable eBPF subsystem
- CONFIG_BPF_SYSCALL to enable the `bpf()` syscall
- CONFIG_BPF_JIT to enable JIT on programs and so increase performance
- CONFIG_BPF_JIT_ALWAYS_ON to force JIT
- CONFIG_BPF_UNPRIV_DEFAULT_OFF=n in **development** to allow eBPF usage without root
- You may then want to enable more general features to "unlock" specific hooking locations:
  - CONFIG_KPROBES to allow hooking programs on kprobes
  - CONFIG_TRACING to allow hooking programs on kernel tracepoints
  - CONFIG_NET_CLS_BPF to write packets classifiers
  - CONFIG_CGROUP_BPF to attach programs on cgroups hooks



eBPF ISA

- eBPF is a “virtual” ISA, defining its own set of instructions: load and store instructions, arithmetic instructions, jump instructions,etc
- It also defines a set of 10 64-bits wide registers as well as a calling convention:
  - `R0`: return value from functions and BPF program
  - `R1, R2, R3, R4, R5`: function arguments
  - `R6, R7, R8, R9`: callee-saved registers
  - `R10`: stack pointer

```
; bpf_printk("Hello %s\n", "World");
      0:  r1 = 0x0 ll
      2:  r2 = 0xa
      3:  r3 = 0x0 ll
      5:  call 0x6
; return 0;
      6:  r0 = 0x0
      7:  exit
```



The eBPF verifier

- When loaded into the kernel, a program must first be validated by the eBPF verifier.
- The verifier is a complex piece of software which checks eBPF programs against a set of rules to ensure that running those may not compromise the whole kernel. For example:
  - a program must always return and so not contain paths which could make them "infinite" (e.g: no infinite loop)
  - a program must make sure that a pointer is valid before dereferencing it
  - a program cannot access arbitrary memory addresses, it must use passed context and available helpers
- If a program violates one of the verifier rules, it will be rejected.
- Despite the presence of the verifier, you still need to be careful when writing programs! eBPF programs run with preemption enabled (but CPU migration disabled), so they can still suffer from concurrency issues
  - There are mechanisms and helpers to avoid those issues, like per-CPU maps types.



Program types and attach points

- There are different categories of hooks to which a program can be attached:
  - an arbitrary kprobe
  - a kernel-defined static tracepoint
  - a specific perf event
  - throughout the network stack
  - an arbitrary uprobe
  - and a lot more, see bpf_attach_type
- A specific attach-point type can only be hooked with a set of specific program types, see bpf_prog_type and bpf/libbpf/program_types.
- The program type then defines the data passed to an eBPF program as input when it is invoked. For example:
  - A `BPF_PROG_TYPE_TRACEPOINT` program will receive a structure containing all data returned to userspace by the targeted tracepoint.
  - A `BPF_PROG_TYPE_SCHED_CLS` program (used to implement packet classifiers) will receive a struct ___sk_buff, the kernel representation of a socket buffer.
  - You can learn about the context passed to any program type by checking include/linux/bpf_types.h



eBPF maps

- eBPF programs exchange data with userspace or other programs through maps of different natures:
  - `BPF_MAP_TYPE_ARRAY`: generic array storage. Can be differentiated per CPU
  - `BPF_MAP_TYPE_HASH`: a storage composed of key-value pairs. Keys can be of different types: `__u32`, a device type, an IP address...
  - `BPF_MAP_TYPE_QUEUE`: a FIFO-type queue
  - `BPF_MAP_TYPE_CGROUP_STORAGE`: a specific hash map keyed by a cgroup id. There are other types of maps specific to other object types (inodes, tasks, sockets, etc)
  - etc...
- For basic data, it is easier and more efficient to directly use eBPF global variables (no syscalls involved, contrary to maps)



The `bpf()` syscall

- The kernel exposes a `bpf()` syscall to allow interacting with the eBPF subsystem
- The syscall takes a set of subcommands, and depending on the subcommand, some specific data:
  - BPF_PROG_LOAD to load a bpf program
  - BPF_MAP_CREATE to allocate maps to be used by a program
  - BPF_MAP_LOOKUP_ELEM to search for an entry in a map
  - BPF_MAP_UPDATE_ELEM to update an entry in a map
  - etc
- The syscall works with file descriptors pointing to eBPF resources. Those resources (program, maps, links, etc) remain valid while there is at least one program holding a valid file descriptor to it. Those are automatically cleaned once there are no user left.
- For more details, see man 2 bpf



Writing eBPF programs

- eBPF programs can either be written directly in raw eBPF assembly or in higher level languages (e.g: C or rust), and are compiled using the clang compiler.
- The kernel provides some helpers that can be called from an eBPF program:
  - `bpf_trace_printk` Emits a log to the trace buffer
  - `bpf_map_lookup,update,delete` *elem Manipulates maps*
  - `bpf_probe_read,write`[*user*] Safely read/write data from/to kernel or userspace
  - `bpf_get_current_pid_tgid` Returns current Process ID and Thread group ID
  - `bpf_get_current_uid_gid` Returns current User ID and Group ID
  - `bpf_get_current_comm` Returns the name of the executable running in the current task
  - `bpf_get_current_task` Returns the current struct task_struct
  - Many other helpers are available, see man 7 bpf-helpers
- Kernel also exposes kfuncs (see bpf/kfuncs), but contrary to bpf-helpers, those do not belong to the kernel stable interface.



Manipulating eBPF program

- There are different ways to build, load and manipulate eBPF programs:
  - One way is to write an eBPF program, build it with clang, and then load it, attach it and read data from it with bare `bpf()` calls in a custom userspace program
  - One can also use `bpftool` on the built ebpf program to manipulate it (load, attach, read maps, etc), without writing any userspace tool
  - Or we can write our own eBPF tool thanks to some intermediate libraries which handle most of the hard work, like libbpf
  - We can also use specialized frameworks like BCC or bpftrace to really get all operations (bpf program build included) handled



BCC

- BPF Compiler Collection (BCC) is (as its name suggests) a collection of BPF based tools.
- BCC provides a large number of ready-to-use tools written in BPF.
- Also provides an interface to write, load and hook BPF programs more easily than using “raw” BPF language.
- Available on a large number of architectures and distributions (but not packaged in Buildroot)
  - On debian, when installed, all tools are named `<tool>-bpfcc`.
- BCC requires a kernel version >= 4.1.
- BCC evolves quickly, many distributions have old versions: you may need to compile from the latest sources



Image credits:
https://github.com/iovisor/bcc

BCC tools

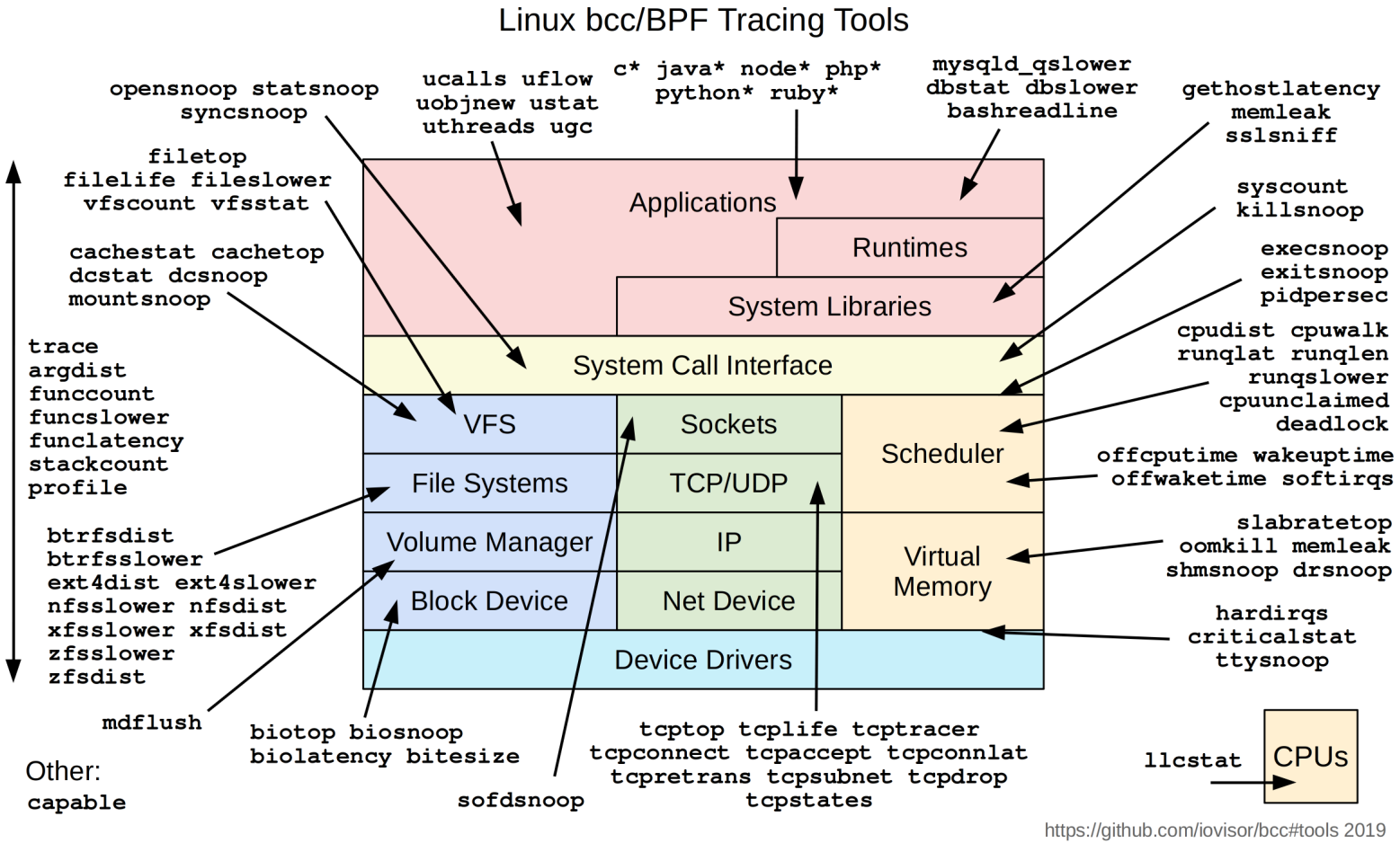


Image credits: https://www.brendangregg.com/ebpf.html



BCC Tools example

- `profile.py` is a CPU profiler allowing to capture stack traces of current execution. Its output can be used for flamegraph generation:

```
$ git clone https://github.com/brendangregg/FlameGraph.git
$ profile.py -df -F 99 10 | ./FlameGraph/flamegraph.pl > flamegraph.svg
```

- `tcpconnect.py` script displays all new TCP connections live

```
$ tcpconnect
PID    COMM         IP SADDR            DADDR            DPORT
220321 ssh          6  ::1              ::1              22
220321 ssh          4  127.0.0.1        127.0.0.1        22
17676  Chrome_Child 6  2a01:cb15:81e4:8100:37cf:d45b:d87d:d97d
2606:50c0:8003::154 443
[...]
```

- And much more to discover at https://github.com/iovisor/bcc



Using BCC with python

- BCC exposes a `bcc` module, and especially a `BPF` class
- eBPF programs are written in C and stored either in external files or directly in a python string.
- When an instance of the `BPF` class is created and fed with the program (either as string or file), it automatically builds, loads, and possibly attaches the program
- There are multiple ways to attach a program:
  - By using a proper program name prefix, depending on the targeted attach point (and so the attach step is performed automatically)
  - By explicitly calling the relevant attach method on the `BPF` instance created earlier



Using BCC with python

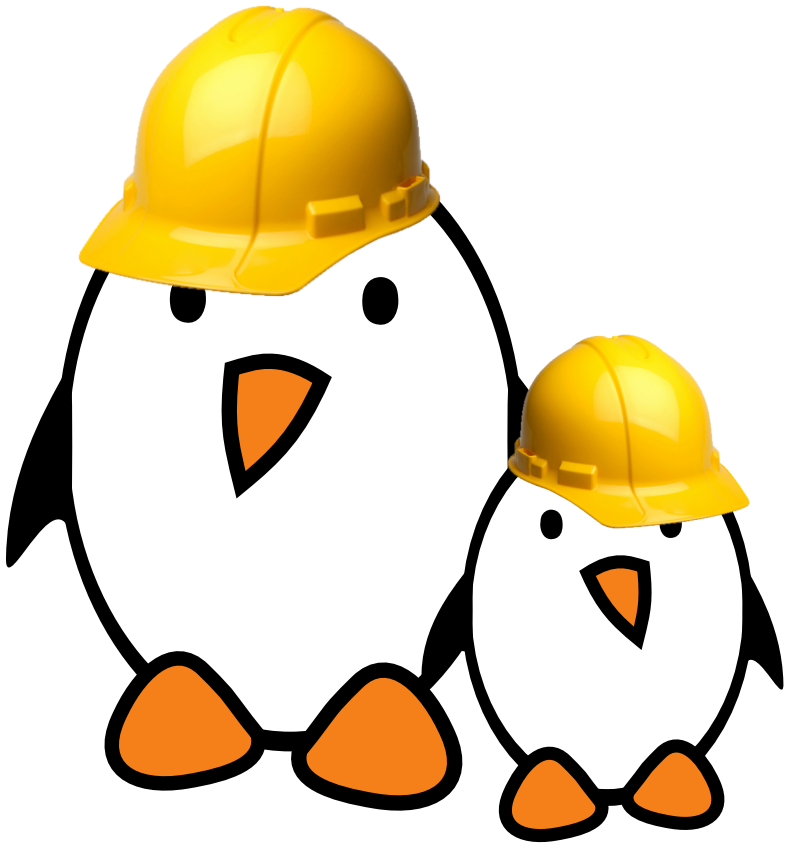
- Hook with a *kprobe* on the `clone()` system call and display `"Hello, World!"` each time it is called

```
#!/usr/bin/env python3

from bcc import BPF

# define BPF program
prog = '''
int hello(void *ctx) {
    bpf_trace_printk("Hello, World!\\n");
    return 0;
}
'''
# load BPF program
b = BPF(text=prog)
b.attach_kprobe(event=b.get_syscall_fnname("clone"), fn_name="hello")
```

Practical lab - Custom eBPF tool with BCC



- Creating custom tracing tools with BCC framework



libbpf

- Instead of using a high level framework like BCC, one can use libbpf to build custom tools with finer control over every aspect of the program.
- libbpf is a C-based library that aims to ease eBPF programming thanks to the following features:
  - userspace APIs to handle open/load/attach/teardown of bpf programs
  - userspace APIs to interact with attached programs
  - eBPF APIs to ease eBPF program writing
- Packaged in many distributions and build systems (e.g.: Buildroot)
- Learn more at https://libbpf.readthedocs.io/en/latest/



eBPF programming with libbpf (1/2)

my_prog.bpf.c

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

#define TASK_COMM_LEN 16
struct {
  __uint(type, BPF_MAP_TYPE_ARRAY);
  __type(key, __u32);
  __type(value, __u64);
  __uint(max_entries, 1);
} counter_map SEC(".maps");

struct sched_switch_args {
  unsigned long long pad;
  char prev_comm[TASK_COMM_LEN];
  int prev_pid;
  int prev_prio;
  long long prev_state;
  char next_comm[TASK_COMM_LEN];
  int next_pid;
  int next_prio;
};
```

- The fields to define in the `*_args` structure are obtained from the event description in `/sys/kernel/tracing/events` (see this example)



eBPF programming with libbpf (2/2)

my_prog.bpf.c

```
SEC("tracepoint/sched/sched_switch")
int sched_tracer(struct sched_switch_args *ctx)
{
  __u32 key = 0;
  __u64 *counter;
  char *file;

  char fmt[] = "Old task was %s, new task is %s\n";
  bpf_trace_printk(fmt, sizeof(fmt), ctx->prev_comm, ctx->next_comm);

  counter = bpf_map_lookup_elem(&counter_map, &key);
  if(counter) {
          *counter += 1;
          bpf_map_update_elem(&counter_map, &key, counter, 0);
  }

  return 0;
}

char LICENSE[] SEC("license") = "Dual BSD/GPL";
```



Building eBPF programs

- An eBPF program written in C can be built into a loadable object thanks to clang:

```
$ clang -target bpf -O2 -g -c my_prog.bpf.c -o my_prog.bpf.o
```

- The `-g` option allows to add debug information as well as BTF information
- GCC can be used too with recent versions
  - the toolchain can be installed with the `gcc-bpf` package in Debian/Ubuntu
  - it exposes the `bpf-unknown-none` target
- To easily manipulate this program with a userspace program based on libbpf, we need "skeleton" APIs, which can be generated with to `bpftool`



bpftool

- `bpftool` is a command line tool allowing to interact with bpf object files and the kernel to manipulate bpf programs:
  - Load programs into the kernel
  - List loaded programs
  - Dump program instructions, either as BPF code or JIT code
  - List loaded maps
  - Dump map content
  - Attach programs to hooks (so they can run)
  - etc
- You may need to mount the bpf filesystem to be able to pin a program (needed to keep a program loaded after bpftool has finished running):

```
$ mount -t bpf none /sys/fs/bpf
```



bpftool

- List loaded programs

```
$ bpftool prog
348: tracepoint  name sched_tracer  tag 3051de4551f07909  gpl loaded_at
2024-08-06T15:43:11+0200  uid 0
xlated 376B  jited 215B  memlock 4096B  map_ids 146,148
btf_id 545
```

- Load and attach a program

```
$ bpftool prog loadall trace_execve.bpf.o /sys/fs/bpf/myprog autoattach
```

- Unload a program

```
$ rm -rf /sys/fs/bpf/myprog
```



bpftool

- Dump a loaded program

```
$ bpftool prog dump xlated id 348
int sched_tracer(struct sched_switch_args * ctx):
; int sched_tracer(struct sched_switch_args *ctx)
  0: (bf) r4 = r1
  1: (b7) r1 = 0
; __u32 key = 0;
  2: (63) *(u32 *)(r10 -4) = r1
; char fmt[] = "Old task was %s, new task is %s\n";
  3: (73) *(u8 *)(r10 -8) = r1
  4: (18) r1 = 0xa7325207369206b
  6: (7b) *(u64 *)(r10 -16) = r1
  7: (18) r1 = 0x7361742077656e20
[...]
```

- Dump eBPF program logs

```
$ bpftool prog tracelog
kworker/u80:0-11  [013] d..41  1796.003605: bpf_trace_printk: Old task was kworker/u80:0, new task is swapper/13
<idle>-0          [013] d..41  1796.003609: bpf_trace_printk: Old task was swapper/13, new task is kworker/u80:0
sudo-18640        [010] d..41  1796.003613: bpf_trace_printk: Old task was sudo, new task is swapper/10
<idle>-0          [010] d..41  1796.003617: bpf_trace_printk: Old task was swapper/10, new task is sudo
[...]
```



bpftool

- List created maps

```
$ bpftool map
80: array  name counter_map  flags 0x0
    key 4B  value 8B  max_entries 1  memlock 256B
    btf_id 421
82: array  name .rodata.str1.1  flags 0x80
    key 4B  value 33B  max_entries 1  memlock 288B
    frozen
96: array  name libbpf_global  flags 0x0
    key 4B  value 32B  max_entries 1  memlock 280B
[...]
```

- Show a map content

```
$ sudo bpftool map dump id 80
[{
  "key": 0,
  "value": 4877514
  }
])
```



bpftool

- Generate libbpf APIs to manipulate a program

```
$ bpftool gen skeleton trace_sched_switch.bpf.o name trace_sched_switch
  > trace_sched_switch.skel.h
```

- We can then write our userspace program and benefit from high level APIs to manipulate our eBPF program:
  - instantiation of a global context object which will have references to all of our programs, maps, links, etc
  - loading/attaching/unloading of our programs
  - eBPF program directly embedded in the generated header as a byte array



Userspace code with libbpf

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include "trace_sched_switch.skel.h"

int main(int argc, char *argv[])
{
    struct trace_sched_switch *skel;
    int key = 0;
    long counter = 0;

    skel = trace_sched_switch__open_and_load();
    if(!skel)
        exit(EXIT_FAILURE);
    if (trace_sched_switch__attach(skel)) {
        trace_sched_switch__destroy(skel);
        exit(EXIT_FAILURE);
    }

    while(true) {
        bpf_map__lookup_elem(skel->maps.counter_map, &key, sizeof(key), &counter, sizeof(counter), 0);
        fprintf(stderr, "Scheduling switch count: %d\n", counter);
        sleep(1);
    }

    return 0;
}
```



eBPF programs portability (1/2)

- Kernel internals, contrary to userspace APIs, do not expose stable APIs. This means that an eBPF program manipulating some kernel data may not work with another kernel version
- The CO-RE (Compile Once - Run Everywhere) approach aims to solve this issue and make programs portable between **kernel versions**. It relies on the following features:
  - your kernel must be built with CONFIG_DEBUG_INFO_BTF=y to have BTF data embedded. BTF is a format similar to dwarf which encodes data layout and function signatures in an efficient way.
  - your eBPF compiler must be able to emit BTF relocations (both clang and GCC are capable of this on recent versions, with the `-g` argument)
  - you need a BPF loader capable of processing BPF programs based on BTF data and adjust accordingly data access: `libbpf` is the de-facto standard bpf loader
  - you then need eBPF APIs to read/write to CO-RE relocatable variables. libbpf provides such helpers, like `bpf_core_read`
- To learn more, take a look at Andrii Nakryiko's CO-RE guide



eBPF programs portability (2/2)

- Despite CO-RE, you may still face different constraints on different kernel versions, because of major features introduction or change, since the eBPF subsystem keeps receiving frequent updates:
  - eBPF tail calls (which allow a program to call a function) have been added in version 4.2, and allow to call another program only since version 5.10
  - eBPF spin locks have been added in version 5.1 to prevent concurrent access to maps shared between CPUs.
  - Different attach types keep being added, but possibly on different kernel versions when it depends on the architecture: fentry/fexit attach points have been added in kernel 5.5 for x86 but in 6.0 for arm32.
  - Any kind of loop (even bounded) was forbidden until version 5.3
  - `CAP_BPF` capability, allowing a process to perform eBPF tasks, has been added in version 5.8



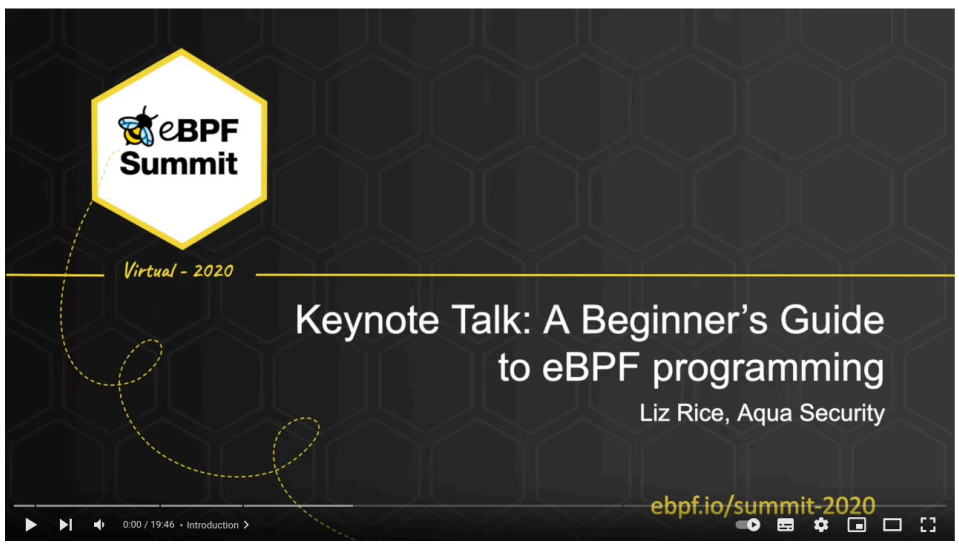
eBPF for tracing/profiling

- eBPF is a very powerful framework to spy on kernel internals: thanks to the wide variety of attach point, you can expose almost any kernel code path and data.
- In the meantime, eBPF programs remain isolated from kernel code, which makes it safe (compared to kernel development) and easy to use.
- Thanks to the in-kernel interpreter and optimizations like JIT compilation, eBPF is very well suited for tracing or profiling with low overhead, even in production environments, while being very flexible.
- This is why eBPF adoption level keeps growing for debugging, tracing and profiling in the Linux ecosystem. As a few examples, we find eBPF usage in:
  - tracing frameworks like BCC and bpftrace
  - network infrastructure components, like Cilium or Calico
  - network packet tracers, like pwru or dropwatch
  - And many more, check ebpf.io for more examples



eBPF: resources

- libbpf-bootstrap: https://github.com/libbpf/libbpf-bootstrap
- A Beginner's Guide to eBPF Programming - Liz Rice, 2020
  - Video: https://www.youtube.com/watch?v=lrSExTfS-iQ
  - Resources: https://github.com/lizrice/ebpf-beginners



Practical lab - Advanced eBPF development



Porting our custom tracing tool for embedded use case

- Converting a BCC script to libbpf
- Bringing advanced features to the tool



Choosing the right tool



Choosing the right tool

- Before starting to profile or trace, one should know which type of tool to use.
- This choice is guided by the level of profiling
- Often start by analyzing/optimizing the application level using application tracing/profiling tools (valgrind, perf, etc).
- Then analyze user space + kernel performance
- Finally, trace or profile the whole system if the performance problems happens only when running under a loaded system.
  - For “constant” load problems, snapshot tools works fine.
  - For sporadic problems, record traces and analyze them.
- If you happen to have a complex setup that you often have to bring up, it is likely a sign that you want to ease this setup with some custom tooling: scripting, custom traces, eBPF, etc